

3교시: 웹 애플리케이션 + DB Compose 실행

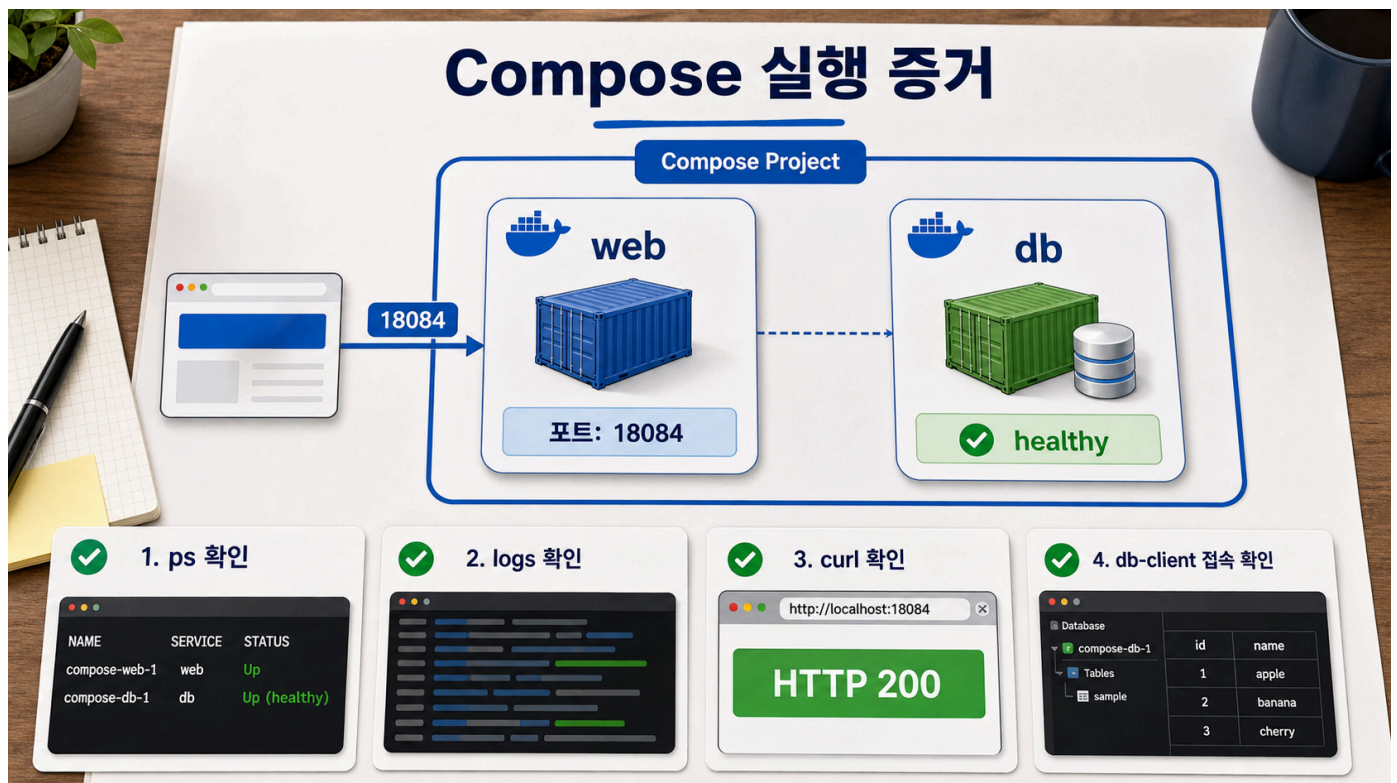
수업 목표

- `docker compose up -d` 로 web과 db를 함께 실행한다.
- `docker compose ps` , `logs` , `run` 으로 정상 evidence를 확보한다.
- HTTP service와 DB service를 각각 다른 기준으로 확인한다.

50분 흐름

시간	활동	비중	산출
0-8분	config 결과 재확인	실행 15%	config evidence
8-20분	<code>compose up -d</code> 실행	실행 25%	project start
20-32분	web HTTP 확인	실행 25%	HTTP evidence
32-42분	db logs와 client 확인	실행 25%	DB evidence
42-50분	evidence 표 작성	실행 10%	run checklist

Visual 1: Compose up 이후 evidence



이 visual은 `up` 이후 확인할 대상을 보여준다. 볼 지점은 web은 HTTP로, db는 readiness/log/query로 확인한다는 점이다.

실행 명령

```
cd week2/day4/labs/compose-app
cp .env.example .env
docker compose config
docker compose up -d
docker compose ps
```

```
curl -I http://localhost:18084
curl -s http://localhost:18084 | grep compose-site-v1
docker compose logs db
docker compose run --rm db-client
```

기대 결과

```
HTTP/1.1 200 OK
compose-site-v1
database system is ready to accept connections
current_database | current_user
paperclip        | paperclip
```

핵심 설명

`docker compose up -d` 는 Compose file에 정의된 service를 project 단위로 시작한다. `-d` 는 detached mode이므로 terminal을 점유하지 않는다. 실행 후에는 반드시 `docker compose ps` 로 실제 상태를 확인한다.

web service의 정상 기준은 HTTP다. container가 running이어도 port가 잘못되었거나 페이지가 기대한 파일이 아닐 수 있다. 그래서 header와 body marker를 함께 확인한다.

db service의 정상 기준은 HTTP가 아니다. PostgreSQL은 `pg_isready` , log, SQL query로 확인한다. Day 4의 `db-client` service는 같은 Compose network 안에서 `db` service name으로 접속하는 증거를 남기기 위해 둔다.

판단 기준

대상	정상 기준	명령
Compose project	service가 running	<code>docker compose ps</code>
web	HTTP 200	<code>curl -I</code>
web body	기대 marker	<code>grep compose-site-v1</code>
db process	ready log	<code>docker compose logs db</code>
db query	database/user 출력	<code>docker compose run --rm db-client</code>

핵심 유의사항

`up` 성공 메시지만으로 완료 처리하지 않는다. image pull은 성공했지만 service가 바로 종료될 수 있고, db가 아직 준비되지 않았을 수 있다. Day 4에서는 `up -> ps -> logs/check` 순서로 evidence를 남긴다.

cleanup

```
docker compose down
```

기록 템플릿

```
## Lesson 3 Compose Run Evidence
- config:
- ps web:
- ps db:
- HTTP status:
```

- body marker:
- db log:
- db-client result:
- cleanup:

평가 기준

기준	2점 evidence
실행	compose up -d 후 상태를 확인했다
web 확인	HTTP header와 body marker를 확인했다
DB 확인	log와 query evidence를 남겼다

공식 근거 링크

- Docker Compose CLI reference: <https://docs.docker.com/compose/reference/>
- PostgreSQL Docker Official Image: https://hub.docker.com/_/postgres

선행 지식과 범위 경계

이 교시는 Compose file을 실제로 실행하고 관찰하는 시간이다. up 을 실행하는 것보다 중요한 것은 실행 후 어떤 증거를 정상 기준으로 삼을지 정하는 것이다.

학생은 container가 running이라고 해서 service가 정상인 것은 아니라는 점을 Day 3에서 경험했다. Day 4에서는 그 원칙을 multi-service project에 적용한다. web은 HTTP evidence가 필요하고, DB는 readiness와 query evidence가 필요하다.

학술 기준 연결

이 교시는 experiential learning 구조에 맞다. 학생은 명령을 실행하고, 결과를 관찰하고, 그 결과를 개념과 연결한 뒤, 다음 실험 또는 실패 재현으로 넘어간다.

단계	Day 4 Lesson 3 활동
Concrete experience	docker compose up -d 실행
Reflective observation	ps, curl, logs, db-client 결과 기록
Abstract conceptualization	web readiness와 DB readiness 차이 설명
Active experimentation	wrong port 또는 missing env drill로 재확인

ABET의 커뮤니케이션 outcome도 포함된다. 학생은 결과를 개인 terminal에만 남기지 않고 README나 evidence note에 재현 가능한 형식으로 써야 한다.

실행 전 preflight

실행 전에 다음 상태를 확인한다.

확인	명령	정상 기준
Docker daemon	docker version	client/server 정보 출력
Compose CLI	docker compose version	version 출력
file 해석	docker compose config	services/networks/volumes 출력
env 값	.env 또는 --env-file	required variable 누락 없음

port 충돌 가능성	docker compose ps 또는 OS port 확인	기존 project와 충돌 없음
-------------	---------------------------------	-------------------

preflight는 시간을 늦추는 절차가 아니라 실패 위치를 좁히는 절차다.

up -d 출력 읽기

Compose는 up 과정에서 network, volume, container를 만든다. 출력은 다음 순서로 해석한다.

- 1. Network creating/created
- 2. Volume creating/created
- 3. Container creating/created
- 4. Container starting/started
- 5. Healthcheck waiting/healthy

DB가 healthy가 된 뒤 web이 시작되는 흐름을 보면 depends_on.condition: service_healthy 가 작동한 것이다. 단, 이것은 DB process readiness를 도울 뿐, 실제 application query retry 정책을 대체하지 않는다.

web evidence 깊이

web service는 세 가지 수준으로 확인한다.

```
docker compose ps
curl -I http://localhost:18084
curl -s http://localhost:18084 | grep compose-site-v1
```

확인	의미
ps	container와 port publish 상태
curl -I	HTTP protocol level status
body grep	기대한 content가 실제 제공되는지 확인

HTTP 200만으로는 부족할 수 있다. reverse proxy나 cache가 다른 페이지를 줄 수도 있기 때문이다. 그래서 marker 문자열을 함께 확인한다.

DB evidence 깊이

DB service는 다음 세 가지를 구분한다.

확인	명령	의미
process log	docker compose logs db	DB server startup 과정
readiness	pg_isready	connection을 받을 준비
query	psql -c ...	실제 authentication과 database 접근

Day 4의 db-client 는 query evidence를 남기는 도구 service다. profiles: ["tools"] 로 기본 실행 대상과 분리되어 있다. 이런 구조는 실무에서도 migration, debug client, admin utility를 평소 service와 분리할 때 사용된다.

실무 관점: healthy의 한계

healthy 는 healthcheck command가 성공했다는 뜻이다. 서비스의 모든 business function이 정상이라는 뜻은 아니다. 예를 들어 DB 가 healthy여도 schema migration이 실패했거나 application user 권한이 부족할 수 있다.

따라서 strong evidence는 다음처럼 여러 층을 가진다.

container state -> port publish -> protocol response -> content marker -> dependency query

관찰 기록 예시

```
## Compose Run Evidence
- command: docker compose --env-file .env.example up -d
- web status: Up, 0.0.0.0:18084->80/tcp
- db status: Up healthy
- HTTP: HTTP/1.1 200 OK
- body marker: compose-site-v1
- DB query: current_database=paperclip, current_user=paperclip
- cleanup: docker compose down -v
```

실무 failure mode

Failure mode	관찰 증상	다음 확인
image 없음	pull 또는 inspect 실패	network, registry, tag
env 누락	config 단계 실패	.env, required variable
port 충돌	web start 실패 또는 bind error	host port 변경
db readiness 지연	db-client 실패	logs db, healthcheck
stale volume	init SQL 반영 안 됨	down -v 위험 확인

오해 점검 문항

문항	기대 답
up -d가 성공하면 모든 기능이 정상인가	아니다. service별 check가 필요하다
web과 DB를 같은 방식으로 확인하는가	아니다. protocol과 readiness 기준이 다르다
db-client는 운영 service인가	아니다. 확인용 tool profile service다
body grep은 왜 필요한가	기대한 파일/content가 제공되는지 확인하기 위해

전이 과제

학생은 자신의 project에 대해 "정상 상태를 증명하는 최소 3개 evidence"를 정한다.

계층	내 project evidence
container 상태	
protocol 응답	
content 또는 dependency 확인	
cleanup 결과	

이 표는 Day 5 발표에서 "실행됩니다"라는 주장 대신 보여줄 증거가 된다.